

# Algorithm for Bioinformatics

## Exercise 6 David Riemer, Usama Mehmood

### Problem 1

Our Algorithm is designed as follows:

Since in this exercise  $m = n$ , we choose our string  $B[m - 1]$  letters and dedine  $B^* = B + B$ . (Meaning that we double B). So we can fix A as a linear string, and only consider rotations of B.

This is true: Lets consider the optimal alignment is some alignment with  $A$  being rotated  $i$  steps to the left and  $B$  being rotated  $j$  steps to the right. This alignment would be the same for  $A^{i-i}$  (which is just the original A) and  $B^{j-i}$ .

In class we discussed Fitting alignment, which allowed the ignorePrefix and ignoreSuffix operations. Every  $A$  can be aligned with a substring of  $B^*$ . We then create the Table of optimal alignment scortes of size  $D[m + 1][2n + 1]$ . The minimum of that table corresponds to the best alignment between  $A$  and some substring  $B^*$ .

### Runtime

Table size:  $(m + 1) \times (2n + 1) = \Theta(mn)$ . Each cell computed in  $O(1)$  from its neighbors. So te total runtime is  $\Theta(n^2)$ , whcih is in  $\Theta(n^3)$ .

### Problem 2

To solve this problem we first look at a simpler case to design our algorithm. We assume we have some string  $S$  of length  $m + p$  and want to check, wether or not this string contains a shuffle of the strings  $P[0..m)$  and  $Q[0..p)$ . We design a matrix of size  $D[m + 1][p + 1]$  that is filled up as follows:  $(i = (0..m), j = (0..p))$ .

$$\begin{aligned}
D[0][0] &= 1 \\
D[0][j] &= \begin{cases} 1, & \text{if } D[0][j-1] = 1 \text{ and } S[j-1] = Q[j-1], \\ 0, & \text{otherwise.} \end{cases} \\
D[i][0] &= \begin{cases} 1, & \text{if } D[i-1][0] = 1 \text{ and } S[i-1] = P[i-1], \\ 0, & \text{otherwise.} \end{cases}
\end{aligned}$$

Along the top row, we haven't used any characters from  $P$  yet, so the first  $j$  characters of  $S$  must all come from the characters of  $Q$  consists of, in the same order. If at any point  $X$  and  $Q$  mismatch, the whole row becomes impossible from there on. The same goes for the first column, just with  $P$  and  $i$ , instead of  $Q$  and  $j$ .

The inner cells are defined with:

$$D[i][j] = \begin{cases} 1, & \text{if } (D[i-1][j] = 1 \wedge S[i+j-1] = P[i-1]) \\ & \vee (D[i][j-1] = 1 \wedge S[i+j-1] = Q[j-1]), \\ 0, & \text{otherwise.} \end{cases}$$

When finishing up with the matrix, the substring  $S$  has a shuffle of  $P$  and  $Q$ , if  $D[i][j] = 1$ .

We can extend the algorithm to run on every single start position of the original Text  $T$  with length  $n$ , choosing a substring of length  $m + p$  for every possibility and creating and filling out the matrix for each, leading to a runtime of  $\Theta(n * m * p)$ . If  $D[i][j]$  is true for any substring, the algorithm returns true terminates. If not it returns false.